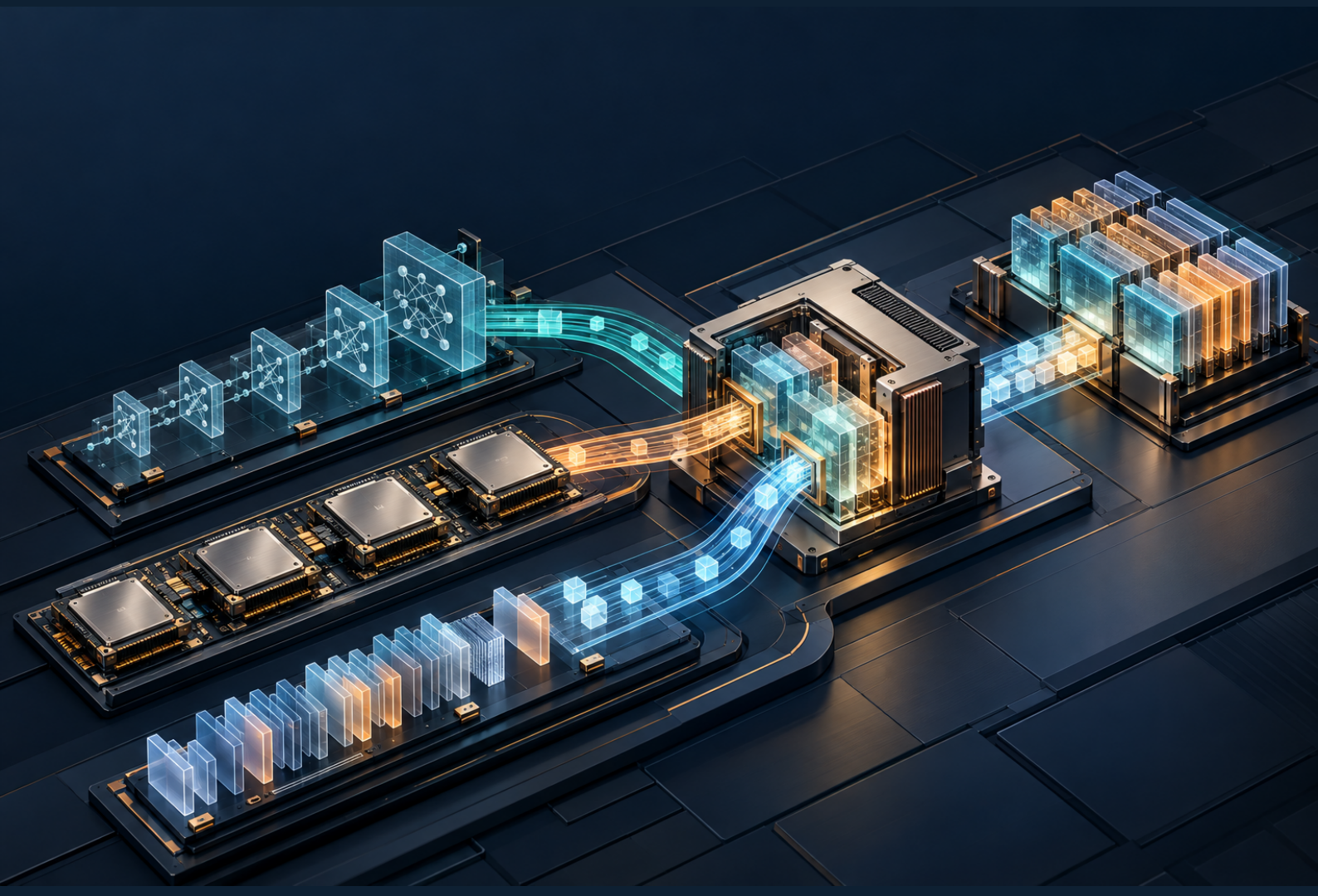
InferenceForge

面向模型与场景的专用推理框架构造

执行反馈驱动代码生成，存储与推理配套设计



作品介绍与技术说明

参赛方向：开源基础软件与解决方案

中国科学技术大学 / 闫超美

yanchaomei@mail.ustc.edu.cn

项目概览：把专用推理引擎做成可交付的软件

企业部署大模型时，真正需要解决的是一个具体问题：选定的模型，放到已有设备上，能否在业务的请求长度、并发量和内存条件下稳定运行。模型参数决定了计算规模，设备决定了带宽与算力，请求形态又决定了批处理和缓存是否值得做。三者共同作用，单独调整某个参数，往往无法解释整条服务路径的表现。

InferenceForge围绕这三个条件构造专用推理软件。使用者提交模型、硬件、负载和约束，Coding Agent据此生成或修改运行时；独立执行器在真实GPU上运行候选，把输出差异、状态问题、资源占用和时延记录交回构造器。通过检查的代码以普通软件形式交付，包括源码、依赖、启动方式和结果记录。业务请求到来后，由这套代码直接执行推理。

项目把存储放在推理路径内共同考虑。会话状态何时留在显存、何时写回主机、恢复后从哪个token继续，这些决定会同时影响数据传输量、计算时间和可容纳的会话数量。我们因此同时开放KV存储组件与消费它的运行时接口，让构造器能够改动配套代码，再用相同请求核对结果。

当前开源交付包含两条完整使用路径。Qwen2.5-0.5B示例提供批量解码和HTTP/SSE服务，适合快速运行、理解构造流程。Qwen3-8B实验提供主机KV保存与恢复、多会话消费者、独立构造入口和四种组件组合的评测，适合研究更大模型上的存储与执行配合。

交付组成	使用者实际获得的能力
专用运行时源码	查看模型前向、批处理、请求状态与存储实现，按场景继续修改
构造工具链	调用Coding Agent，保存提示、代码快照、调用用量与失败记录
独立执行环境	在真实GPU上检查输出、状态、资源与时间，形成可定位的反馈
可复现结果	按固定模型、设备和请求重跑，回读对应版本的原始记录
开源交付	MIT代码、运行指南、原始结果与验证记录、版本发布与演示入口

本作品面向AI基础设施研发、专用推理部署和系统教学。项目的积累点，是把一次调优过程变成可以继续运行、检查和复用的软件资产。

来源：公开版本v0.2.0；README.md、docs/ARCHITECTURE.md、experiments/qwen3_8b/README.md。

阅读导航与作品结构

这份说明按“为什么做、怎样运行、如何验证、怎样使用”展开。前半部分说明项目要解决的部署问题和构造方法；中间部分进入请求执行、KV存储与会话状态；后半部分给出实验、复现和开源交付。

阅读部分	主要内容	页码
项目与场景	部署矛盾、特化对象、产品定位、应用方式	4—8
构造体系	总体架构、任务合同、Agent工作区、执行环境	9—12
执行反馈	检查顺序、数值语义、返回信息	13—14
推理运行时	服务调度、批量解码、状态与文本流	15—17
存储与会话	KV规模、保存恢复、接口、增量与驱逐	18—22
实验结果	评测设计、0.5B构造、8B恢复、会话组合	23—27
工程质量	故障检查、版本与交付结构	28—29
上手操作	启动推理服务、复现8B实验	30—31
开源与发展	贡献方式、核心特点、后续工作	32—33
提交入口	代码、文档、视频、版本与参考资料	34

面向不同读者的建议

评审可先读项目概览、总体架构、三组实验与交付入口，形成对作品价值和完成度的整体判断。基础设施研发人员可重点阅读任务合同、固定评测、KV接口和会话消费者，了解哪些部分能够替换、哪些状态需要共同维护。准备实际运行的使用者，可直接按第30—31页的步骤进入公开仓库。

本文采用的术语

“构造器”指开发阶段调用Coding Agent的控制程序；“候选”指一次生成后待评测的源码快照；“运行时”指真正处理请求的推理和状态代码；“固定评测”指从受信起点装配的检查程序。“存储”在当前8B实现中具体指GPU与主机锁页内存之间的KV状态保存和恢复。

正文中的实验图根据公开JSON结果绘制。架构与机制插图用于说明组件关系；源码、数据、插图制作记录和最终文件校验值在交付目录中分别保存。

01 从部署条件出发理解推理效率

同一个模型，处理短问答和长文档时，瓶颈并不相同。短请求往往频繁经历调度、输入准备和少量解码；长输入会增加预填充计算；多轮会话则需要保留已经计算过的上下文。即使设备相同，输入长度、输出长度和会话重访方式发生变化，适合的执行策略也会改变。

通用推理软件需要照顾大量模型结构、硬件能力和接口形态。这种覆盖范围具有实际价值，同时也增加了专门调优时需要理解的代码和配置。对某个固定模型、固定设备和较稳定负载，开发者真正关心的往往只是其中少数路径：权重如何读取，哪些请求可以合批，KV放在哪里，下一轮请求到来时从什么状态继续。

过去，这些工作主要依赖熟悉模型和系统的工程师逐层定位。一次性能调整可能同时触及算子调用、张量布局、缓存语义和调度器。代码写出来以后，还需要判断输出是否改变、显存有没有增加、并发请求会不会互相覆盖。调优成本由“理解与修改”以及“执行与验证”共同组成。

Coding Agent降低了修改代码的门槛，但能生成代码并不自动解决执行判断。若环境只返回“运行成功”或一个吞吐数字，构造器很难知道性能变化来自何处，也难以定位跨请求状态或恢复位置错误。InferenceForge因此把真实执行环境作为产品的核心：让每次代码修改尽快得到具体、可比较、能够指导下一步的结果。

一个常见的部署问题

设想某个内部助手使用固定8B模型。白天会有多个活跃会话，部分用户几秒后继续提问，另一些会话暂时不再访问。如果所有历史KV都留在显存中，容量会越来越紧；如果每次都重新计算历史，又会重复消耗预填充时间。合适的实现需要根据访问规律决定保存、恢复与重算，且保证恢复后的生成状态与连续推理一致。

项目选择从这类可以明确输入、输出与资源边界的任务切入。场景越具体，构造器就越容易获得稳定的评价标准，生成产物也越容易被实际使用者检查和复现。

来源：docs/SPECIALIZATION.md；docs/ARCHITECTURE.md。

02 特化对象：模型、硬件与请求形态

InferenceForge的输入不是一句“把推理做快”。一个可执行的特化任务必须说明，哪些条件固定，哪些部分允许修改，以及最终用什么负载评价。模型结构、硬件资源和请求形态是任务的三个基本维度。

模型决定计算和状态规模

层数、隐藏维度、注意力头数、KV头数、词表大小以及数据类型，决定了前向计算和会话状态的基本形态。构造器可以利用这些已经固定的信息组织代码。例如，当前8B实例固定为Qwen3结构和BF16权重，KV张量形状与层数可以直接进入存储接口和预算计算。

硬件决定代价

同样一份KV状态，在设备内重排和通过主机链路往返，代价不同。GPU显存带宽、主机内存类型、数据拷贝接口和可用显存共同影响策略。当前实验以单A40为执行底座，主机锁页内存用于GPU与CPU之间的状态传输。对于另一种设备，首先要重新确认这些代价，再选择适配实现。

负载决定值得优化的路径

输入长度决定预填充规模，输出长度决定解码持续时间，并发量决定合批机会，会话重访和新增输入长度决定缓存的复用程度。任务中固定这些条件，才能区分代码变化带来的收益和请求本身变容易的影响。

条件	当前任务如何表达	可以推动的实现选择
模型身份	仓库revision、配置与权重哈希	权重装载、模型前向、精度与形状
硬件身份	设备标识、软件环境、资源预算	批量大小、内存分配与拷贝方式
请求形态	输入、输出长度与固定请求序列	合批、状态保留、驱逐与恢复
输出语义	token、概率、文本和KV续写要求	数值实现、状态接口与采样路径
优化空间	可编辑源码和必需接口	引擎、存储与消费者的具体修改

这些条件形成一个可以执行的任务合同。合同使构造器知道目标，使评测器知道检查什么，也使后来使用代码的人知道产物适用于哪类工作。

来源：tasks/qwen25_05b/task.json；targets/qwen3-8b-a40/target.json；8B存储与会话合同。

03 产品定位与交付方式

InferenceForge交付一套构造工具链，也交付由它产生的可运行代码。二者分别满足不同的使用需求：研发团队可以继续探索自己的特化任务；应用团队可以先运行已提供的引擎，再根据实际负载决定是否进入构造流程。

构造工具链

控制器负责读取任务配置、准备候选目录、调用Coding Agent、装配固定评测代码和提交远程执行。每次调用都留下输入提示、运行事件、使用量、候选源码和评测结果。失败候选同样保留，使后续修改能够针对实际错误，而不是重新猜测上一轮发生了什么。

独立运行时

生成后的引擎以普通Python模块存在。0.5B示例包含模型前向、KV状态、批量解码和HTTP/SSE服务；8B实验把模型引擎、主机KV存储和会话消费者拆成明确的接口。在线执行不依赖生成器持续连接，构造过程的模型调用也不会进入每个请求的推理时延。

结果与使用说明

代码与支持范围一起交付。用户能够查看模型版本、数据类型、上下文范围、依赖组合和启动命令；实验记录则说明结果来自什么设备、请求和重复方式。这样，一套专用实现既可以作为部署起点，也可以作为下一轮研发的种子。

典型使用者	首先使用的部分	形成的工作产物
推理部署工程师	已生成引擎与快速运行指南	可运行服务、请求记录、场景适配任务
AI基础设施研发	构造器、固定评测和候选接口	新运行时、失败定位、同条件对照
存储系统研发	8B KV组件与会话实验	存取策略、预算行为、恢复成本
高校实验教学	显式前向实现与可读源码	从模型计算到系统调度的完整实验

这种交付方式保留了专用软件的直接性。用户拿到能阅读、能运行、能继续修改的源码，可以纳入自己的研发和部署流程。

来源：README.md；engines/README.md；docs/QUICKSTART.md；MIT LICENSE。

04 应用场景：多轮知识助手的状态复用

企业知识助手通常不是一次问完就结束。用户先提供一段材料，再追问其中的条款、依据和处理方式。前几轮已经读取的内容，在后续问题中仍然有用。系统若能保留相应状态，就可以把计算集中在新增输入和新的输出上。

这类场景首先需要区分“模型权重”和“会话状态”。权重描述模型本身，在多个会话间共享；KV是某个会话已经处理过的token所形成的状态。两类数据的生命周期不同：权重长期驻留，会话KV则随访问和资源压力变化。把这一区别落实到代码中，才能正确决定保存对象和释放时机。

InferenceForge的8B会话接口围绕独立会话组织工作。调用方使用会话标识提交新增输入，消费者维护该会话的KV、下一步logits和待消费token。显存预算允许时，状态留在GPU；需要为其他会话腾出空间时，消费者将完整可恢复状态提交给主机存储，再释放GPU侧会话对象。

一次会话的实际过程

第一轮先完成预填充并生成回答。用户稍后追问时，若状态仍在GPU，消费者继续使用；若此前被驱逐，则从主机恢复。消费者还需要把上轮最后一个尚未进入KV的输出token与本轮新增输入一起前向，才能与连续推理保持一致。这个位置关系是恢复实现中的关键细节。

业务事件	系统动作	对实现的要求
新会话到达	建立状态并预填充	独立会话标识和独立KV
同一会话继续	消费待处理token与新增输入	正确维护KV末位置
显存预算紧张	选择会话、写回并释放	写回完成与释放顺序明确
已驱逐会话重访	恢复状态并继续解码	恢复后的输出与参考一致
会话结束	删除主机与GPU状态	容量记账随生命周期回落

这些步骤不依赖某个具体行业文本。知识库问答、内部资料分析和多轮编程助手，都可以按相同状态问题建立特化任务，再换入各自的请求轨迹。

来源：engines/qwen3_8b_sessions/consumer.py；experiments/qwen3_8b/session_matrix.py。

05 应用场景：固定模型的推理与研发平台

另一类需求来自模型和设备相对固定的服务。团队已经选定模型版本，也有明确的GPU配置，业务请求大多落在一组可观察的长度和并发范围内。这时，优化工作可以围绕真实使用路径展开，而不必同时覆盖所有可能的模型与任务。

当前0.5B示例展示了这种特化方式：原运行时逐请求推进解码，构造器根据并发任务修改了引擎与服务，让活跃请求在同一解码步骤中合批。服务仍然维护每个请求的独立输出和KV状态，客户端继续使用HTTP或SSE接口。改动集中在计算组织方式，对调用方保留了清晰的输入输出形式。

对于基础设施研发平台，项目提供的是一种更短的实验路径。开发者可以先提出可检查的优化目标，允许构造器编辑少数运行时文件，再由固定评测器判断输出、资源和时间。一次试验结束后，源码和结果直接成为下一次工作的起点，便于比较不同实现，而不是只留下人工调参记录。

面向部署的适配顺序

首先收集代表性请求，确定输入长度、输出长度和并发形态。其次固定模型与设备，把正确性要求写进任务。然后从可运行种子开始生成候选，在同条件下检查和测量。最后把通过的代码作为版本化产物交付，并在负载改变时重新评估。

场景特征	优先观察	可形成的任务
大量短请求	调度开销与合批机会	调整活跃请求的解码组织
多轮且经常重访	历史状态大小与复用间隔	配套生成保存、恢复与消费逻辑
输出较长	连续解码时间与状态增长	优化内层执行和KV追加路径
设备或模型变化	算术、接口与资源差异	重建合同并重新验证候选

当前公开运行底座为NVIDIA A40，后续硬件适配沿同一任务方式开展。设备能力进入任务与评测后，构造器才有条件判断哪一种专用实现更合适。

来源：examples/generated/batched/；results/campaign-20261030.json；docs/SPECIALIZATION.md。

06 总体架构：构造阶段与运行阶段

系统分为构造阶段和运行阶段。构造阶段将任务、代码生成和GPU执行连接起来；运行阶段接收应用请求，执行已经生成并选定的引擎。两者通过版本化源码和接口合同衔接。

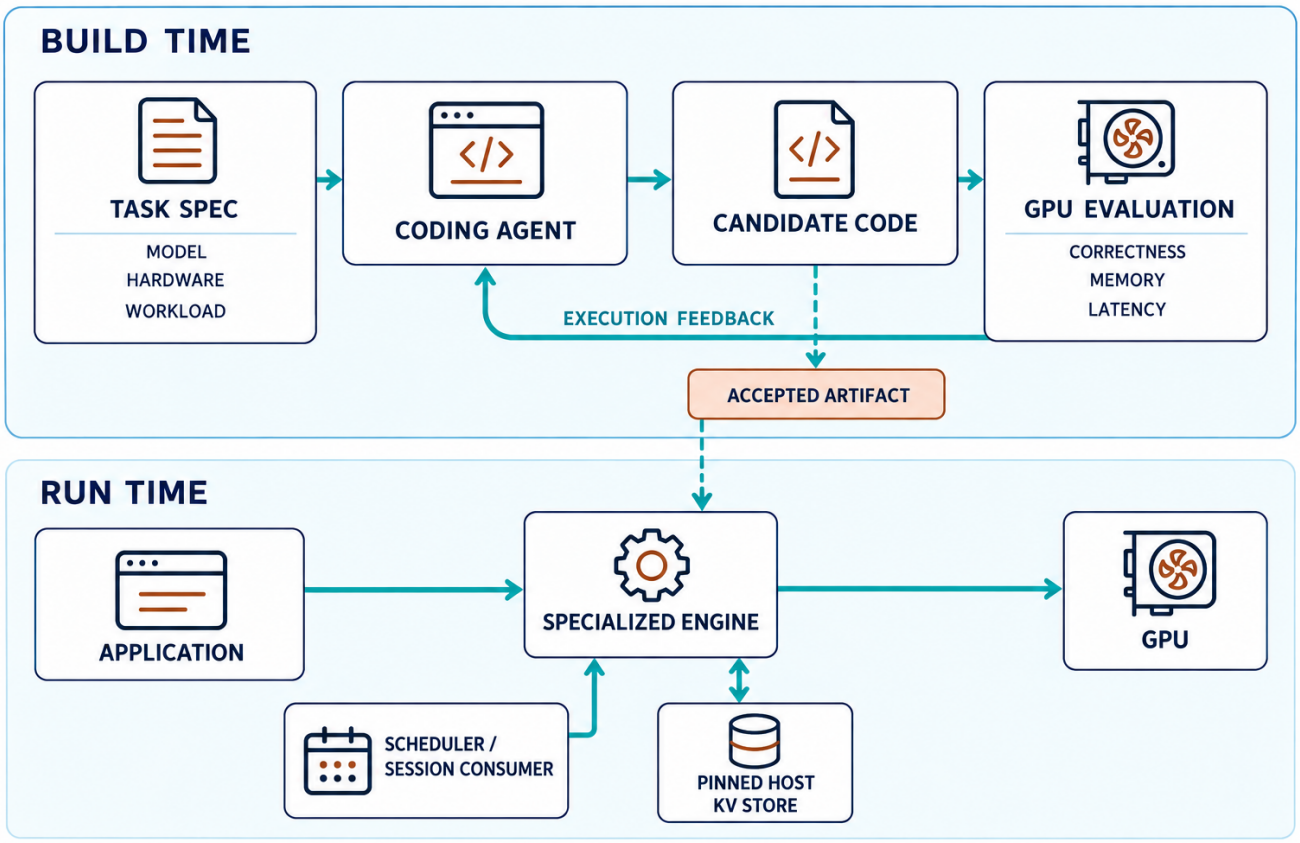


图1 构造工具链与生成运行时的关系。上层完成候选构造和检查，下层展示独立运行的软件组件。存储与消费者可以根据任务组合；当前0.5B服务和8B会话实验分别提供相应实现。

构造阶段的职责

控制器保存受信任务和评测程序，Coding Agent在候选工作区中修改运行代码。候选与固定评测器装配成执行快照后，发送到GPU环境。执行结果回到控制器，成为选择下一轮起点和组织反馈的依据。最终交付需要绑定实际源码，避免把某次测量误归到另一版代码。

运行阶段的职责

模型引擎负责前向计算，会话或请求消费者负责推进状态，存储组件负责保存和恢复。应用只调用这套确定的运行代码。生成器可以继续离线改进新版本，已交付版本的运行不会等待生成器回答。

来源：forge/campaign.py；forge/pilot.py；engines/README.md。

07 任务合同：把优化目标写成可执行条件

任务合同首先解决“测的是同一件事”这个问题。若候选更换权重、减少输出长度或使用不同精度，速度就不能直接归因于实现改进。因此，合同需要固定模型身份、计算精度、输入输出和允许修改的文件。

0.5B任务配置中保存了模型类型、权重SHA256、float32精度、GPU标识、随机种子、最大上下文、输出长度、测试提示和数值误差阈值。路径与设备通过环境变量传入，避免把开发者的机器信息写进公开任务。8B目标进一步记录具体revision及全部模型文件身份，评测前逐文件核对。

约束需要落实到程序

文字说明提供意图，执行器提供实际判定。例如，“恢复后结果正确”需要落到保存何种张量、恢复哪一步状态、以哪个参考进行比较；“遵守预算”需要说明统计已提交载荷、活跃GPU张量还是其他资源。项目将这些要求分别放进任务、接口合同与评测脚本，让构造器能够读取明确条件。

合同内容	典型字段或接口	执行时的作用
模型身份	model revision、文件哈希	在执行前确认输入资产
计算设置	dtype、model_type、context	限定前向与状态形式
请求工作量	prompts、max_new_tokens、trace	保持候选间工作量一致
行为要求	输出、缓存、文本与流式语义	判断结果是否符合预期
编辑范围	engine、server或存储/消费者	约束候选能改动的部分
预算与记录	超时、容量、源码摘要	控制试验成本并保存身份

合同也便于任务迁移。更换模型时，重新固定模型与数值设置；更换业务时，重新固定请求与会话轨迹；更换设备时，重新确认资源和环境。这样，每次适配都有明确的起点和检查方法。

来源：tasks/qwen25_05b/task.json；forge/task.py；experiments/qwen3_8b/host-store-contract.md、session-contract.md。

08 构造器：让每一次修改都有上下文

构造器承担组织工作的职责：准备种子、说明任务、调用Agent、接收候选、执行评测和整理反馈。这里最重要的对象是一次具体试验，而不是一段脱离运行环境的代码回复。

0.5B构造入口允许Agent编辑引擎与服务文件。控制器把这些候选文件与受信代码组装到单独目录，再由GPU执行器运行。8B构造入口则准备存储组件和消费者的种子、合同及输出位置。两条路径都把生成结果保存为实际文件，后续检查面对的就是这些文件。

候选工作区保留什么

每轮保存提示、原始生成事件、源码快照、调用用量和执行结果。若生成器正常退出但没有改变源码，8B入口会记录相应状态；若已经生成代码但尚未执行，记录为待验证产物。这样，生成、执行和通过检查能够对应到真实步骤。

失败如何进入下一轮

有用的反馈应指出错误发生的位置和上下文。例如，缓存续写在第几个token与参考不同，两个请求交错后哪一份状态发生变化，恢复耗时中数据传输占多少。构造器从这些结果组织下一次任务，使Agent集中修改相关路径。

文件或记录	为什么保存
prompt与合同	还原生成器看到的目标与限制
生成事件JSONL	查看实际执行了哪些命令和修改
usage与墙钟时间	统计构造成本，而不仅统计运行速度
候选源码快照	将输出和性能绑定到具体代码
固定评测结果	保留通过、失败与失败发生的位置
远程任务身份	在中断后确认任务位置和完成状态

构造器的效率由两部分决定：它生成代码的能力，以及执行环境提供反馈的速度和质量。项目把后一部分实现为可重复使用的工具链，让新的场景任务能够沿用已有执行与记录机制。

来源：forge/campaign.py；experiments/qwen3_8b/construct.py。

09 执行环境：生成后必须能够实际运行

InferenceForge采用本地构造、远程GPU执行的工作方式。本地负责编辑、调用生成器和保存版本；GPU服务器负责模型执行、检查与测量。两端通过确定的源码快照和结果目录衔接。

固定评测代码从受信起点装配，候选工作区提供允许修改的运行时文件。评测使用原始模型资产，按任务加载参考和候选。0.5B完整验收将参考与候选安排在独立进程中执行，分别产生输出，再对相同请求进行比较。这样的组织方式让参考的状态不依赖候选内部实现。

执行前

执行器确认模型文件和设备条件，记录源码身份和任务参数。模型哈希用于识别权重，代码哈希用于识别实现，软件环境用于解释执行条件。输出目录与试验一一对应，便于在失败后查看原始记录。

执行中

先检查数值和状态行为，再运行预热与测量。对服务路径，检查普通输出、流式token、文本拼接以及请求交错；对8B存储路径，检查保存和恢复的完整张量与后续输出；对会话路径，检查多轮续写和容量行为。检查内容随任务变化，身份与记录方式保持一致。

执行后

候选源码、检查结果和计时数据一起返回。构造器根据结果决定后续工作，使用者也可以跳过自动生成，直接用同一执行入口验收已有候选。独立执行入口使工具链能够服务于Agent生成代码，也能服务于工程师提供的实现。

执行产物	包含内容
身份记录	模型文件、源码摘要、设备与依赖
数值记录	token、概率、文本或完整logits摘要
状态记录	KV长度、会话续写、保存恢复和释放
资源记录	容量、活跃状态与峰值统计
时间记录	加载、构造、预热、执行和重复测量

来源：forge/pilot.py；forge/evaluate.py；experiments/qwen3_8b/identity.py与评测入口。

10 密集反馈：让错误能够指导修改

一次“失败”只有携带上下文，才能有效指导下一次构造。InferenceForge将反馈组织为输出、状态、资源与执行代价四类，让构造器能够区分数学问题、生命周期问题和性能问题。

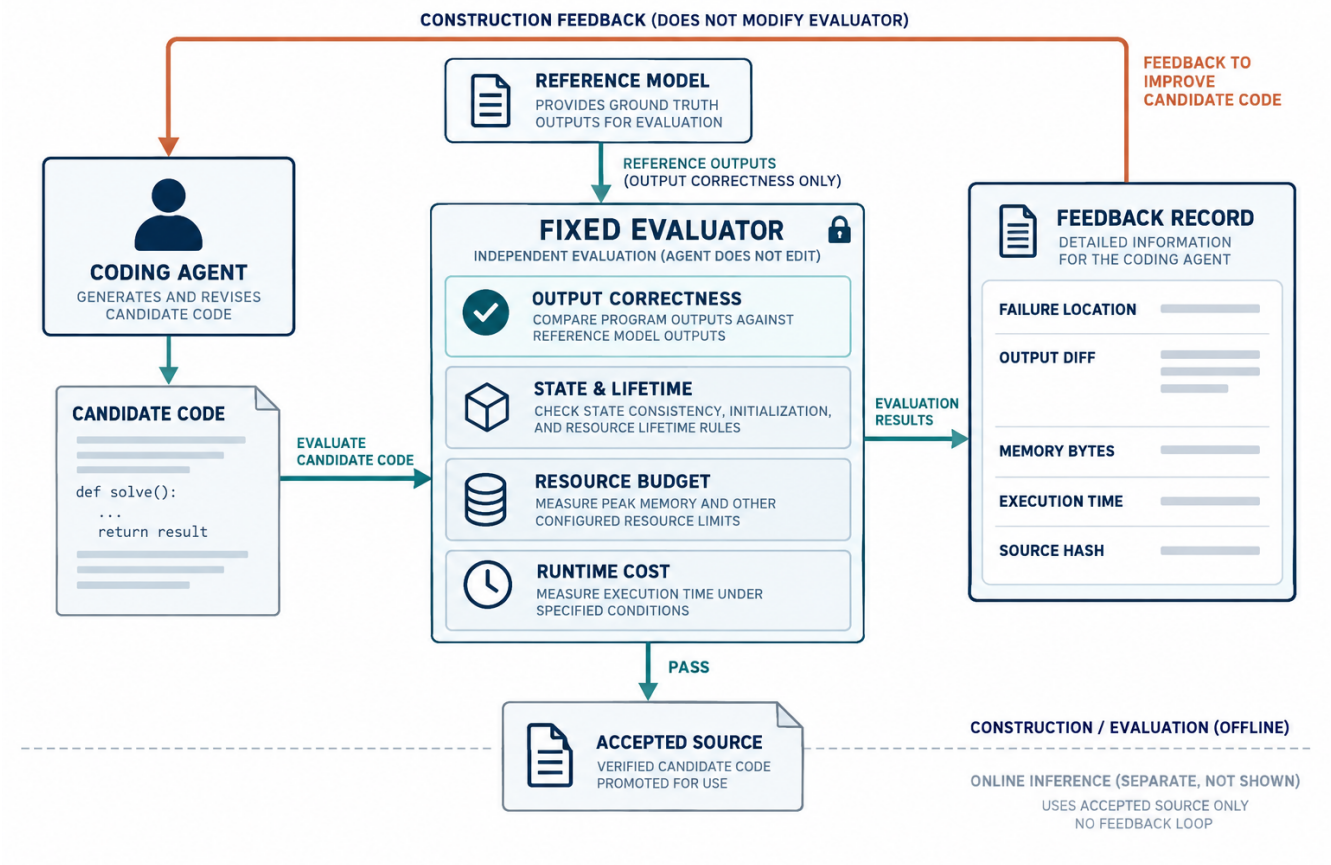


图2 固定评测将候选运行结果整理为具体反馈。参考模型提供输出依据，状态与资源检查约束实际执行，源码摘要连接代码和结果。

反馈的先后关系

首先确认输出能否对应参考。随后检查状态是否能够持续推进，多个请求和多轮会话是否保持独立。满足行为要求后，再解释资源和时间数据。这样的顺序使优化工作有清楚的基础：同样的任务、同样的结果，对应怎样的资源成本。

反馈怎样变得更有用

比起只返回异常栈，记录失败请求、执行步骤、状态长度和相关源码更容易定位问题。比起只给一个总时延，把加载、恢复、继续生成和预热分开，能够帮助构造器判断修改是否落在主要开销上。项目中的失败保留、请求缩小和逐输出记录，均围绕这一目的设计。

来源：forge/evaluate.py、forge/replay.py；8B host_roundtrip.py与session_matrix.py。

11 正确性：从输出词到完整状态

大模型推理的正确性包含多个层次。生成的词是否一致，是调用者最直接看到的结果；词的概率、缓存中的位置和流式文本的拼接，则决定了后续行为是否仍然一致。InferenceForge按这些层次组织检查。

数值检查

0.5B检查使用独立参考模型，比较相同输入下的词表输出、greedy token与指定token概率。固定任务中给出的误差门槛由评测器读取。发布验收还在不同历史长度上进行四步批量续写，将每一步的完整词表结果与参考进行比较，而不是只看一条最终文本。

8B存储实验的目标是检查状态保存和恢复是否改变已经冻结的引擎行为。它在相同模型前向下比较完整KV、下一步logits，以及恢复后实际生成的token。会话矩阵还逐步比较选中token的logprob。这样，存储组件的错误可以与模型算术本身的差异分开定位。

状态检查

返回状态需要有清晰的所有权。保存以后，修改原GPU对象不应改变已经提交的主机快照；恢复得到的对象也不能因下一次恢复而被覆盖。会话继续生成时，还需要检查KV长度、最后一个输出token的位置以及会话之间的隔离。

检查层次	观察对象	典型错误
数值	完整词表、选中token概率	精度或计算形状改变导致差异
输出	token序列、生成长度	少生成一步、重复输出或漏输出
状态	KV张量、logits、位置	保存不完整、恢复位置偏移
所有权	原对象、快照、恢复副本	共享缓冲被后续请求覆盖
协议	SSE事件、文本、结束标志	字符碎片、流式与普通结果不同

这种分层检查使优化过程更容易解释。某个候选即使能够启动，也必须回答“输出对不对、状态能否继续、请求之间是否独立”这些实际问题，随后才进入性能判断。

来源：tests/test_engine.py；tests/check_generated_batch.py；8B host_roundtrip.py。

12 服务路径：请求调度与HTTP/SSE

公开0.5B服务使用HTTP接收请求，推理计算由一个持续运行的调度线程推进。网络处理和模型执行分开，使多个客户端能够并发提交请求，同时让GPU执行保持清楚的状态顺序。

请求到达后，服务解析prompt或input_ids，检查长度和参数类型，为请求分配唯一标识，并把任务放入等待队列。每个请求保存输入token、输出上限、截止时间、消息队列、取消标志和当前引擎状态。调度器接收新任务、推进活跃请求，并将逐token结果写回各自的消息队列。

普通HTTP请求等待最终消息，返回完整文本、token_ids和usage。SSE请求则持续读取消息队列，按token事件发送增量文本，最后发送结束消息和[DONE]。两种返回形式使用同一生成状态，便于核对流式和普通输出的一致性。

接口与资源规则

路径或参数	当前行为
GET /health	返回调度器就绪状态及模型身份
GET /v1/models	返回当前模型描述
POST /v1/completions	接收prompt或input_ids，支持stream参数
上下文与输出	总长度最多512 token；输出1—64 token
请求管理	最多32个在途请求；检查重复request_id
生命周期	每请求有截止时间，断开后设置取消标志

服务把接收、调度、逐token和完成事件写入日志。研发人员因此可以查看请求是否真正发生交错、输出数量是否达到要求、某个错误对应哪个请求。事件日志同时是演示和问题复现的直接依据。

当前服务监听回环地址。远程使用时，通过SSH端口转发连接GPU主机上的服务，模型权重和推理过程保持在该主机内。生成器只参与开发阶段，服务启动后按普通程序运行。

来源：examples/generated/batched/server.py；docs/QUICKSTART.md。

13 批量解码：合并计算，保留请求状态

0.5B实例中的一次实际构造，把逐请求推进的解码方式改成了批量推进。构造器同时修改引擎与服务：引擎增加batch_step接口，服务将可以继续生成的活跃请求组织为一批，再把结果分配回各自请求。

这里的关键不只是把多个token堆成一个张量。不同请求已经处理的历史长度可能不同，KV状态也属于不同请求。合批时必须保留这些边界，让每个请求只访问自己的有效历史，并继续使用自己的绝对位置。

当前实现的组织方式

引擎读取各请求的状态长度，计算当前批次的最大历史长度。每层先为该请求追加新的K/V，再把不同长度的序列补齐到统一形状。注意力掩码将补齐位置排除，位置编码根据各请求实际长度计算。完成当前步后，状态按请求拆开返回。

步骤	执行动作	需要保持的关系
收集活跃请求	提取待输入token和已有状态	token与请求标识一一对应
组织每层KV	追加当前token，再补齐长度	每个请求保留自己的有效长度
批量前向	执行矩阵计算与带掩码注意力	补齐位置不参与有效注意力
拆分结果	返回每请求的新logits和KV	调用者收到独立的状态视图
推进服务	发出token事件或完成消息	只更新对应请求的输出计数

在发布复验中，使用长度3、17和31的历史状态，连续进行四步批量续写，共检查12次完整词表输出，并确认原状态长度保持不变。这个测试直接覆盖了不同长度合批和旧状态保留两个关键行为。

从构造角度看，这个例子说明接口与执行方式可以一起修改：服务提供可合并的活跃请求，引擎提供匹配的批量操作，固定评测再检查这种配合是否保持了原来的请求语义。

来源：examples/generated/batched/engine.py的batch_step；发布验证记录。

14 文本流：token边界与字符边界

流式推理不是把每个token单独转成字符串后拼起来。中文、emoji和其他Unicode字符可能由多个字节片段组成，token边界不一定等于字符边界。如果每个token都立即单独解码，客户端可能看到替换字符，最终文本也可能与普通请求不同。

InferenceForge的服务保留每个请求已经生成的token列表和已经发出的文本。新token到来后，先解码完整token前缀，再确定相对已发送文本新增的稳定部分。中间仍不完整的字符片段暂时不发出，最终事件则完成尾部解码。

一个容易被忽略的细节

某一步确实生成了token，但这个token暂时没有形成完整可显示字符。服务仍然保留该token事件，文本增量可以为空。这样，token计数、索引和最终文本可以同时保持一致，客户端也不会因为没有新文字而误认为模型没有推进。

对象	服务如何维护	校验方式
token序列	逐步追加并记录索引	流式与普通请求比较token_ids
已发出文本	保存累计稳定前缀	新文本必须延续既有前缀
字符碎片	中间步骤暂缓显示	中文和emoji边界回归
最终文本	完整解码所有输出token	与普通请求完整文本一致
结束状态	最终消息后发送[DONE]	检查长度、usage与结束标记

错误处理也需要保持消息顺序。请求超时或推理出错时，服务发送包含错误的结束消息；连接断开后，取消标志交给调度器处理。实际工程中，这些行为决定了上层应用能否可靠地结束一次请求、显示完整文本并回收状态。

v0.2.0发布验收使用中文提示，实际检查了普通输出与SSE输出的token序列和拼接文本相同，同时检查四个并发请求各自返回16个token。验收针对真正启动的服务路径执行。

来源：examples/generated/batched/server.py的incremental_decode；tests/test_engine.py；results/release-v0.2.0-validation.json。

15 KV存储：从容量问题到执行问题

模型权重通常是推理服务中最大的固定资产，KV状态则随请求增长。会话越长、同时保留的会话越多，KV占用就越大。它既是容量问题，也是执行问题：保存和恢复会消耗时间，重算会消耗计算，而恢复位置错误会改变下一步输出。

在本项目冻结的Qwen3-8B配置中，共有36层、8个KV头，每头维度128，KV使用BF16。每个token的K和V载荷可按下式计算：

$2 \times 36 \times 8 \times 128 \times 2$ bytes = 147,456 bytes = 144 KiB

这个数值来自配置与张量维度的计算。它适合估算会话状态规模，实际运行还需要考虑下一步logits、对象管理、临时张量和分配器行为。

已缓存token数	单会话KV载荷
128	18 MiB
512	72 MiB
1,024	144 MiB
2,048	288 MiB
3,072	432 MiB
4,096	576 MiB

为什么只优化拷贝还不够

保存和恢复的时间，最终要放回会话流程中判断。如果恢复只占“恢复后继续生成”总时间的一小部分，即使拷贝本身明显加快，完整请求时间的改善仍可能有限。另一方面，如果每一轮都重复备份已经保存过的前缀，减少同步次数与减少数据量可能比微调单次拷贝更直接。

因此，项目同时提供存储组件和消费者接口。存储组件决定怎样保存；消费者决定何时保存、何时恢复以及恢复后怎样继续。这两部分共享正确的状态语义，才能共同影响完整执行路径。

来源：experiments/qwen3_8b/model.json中的冻结配置；engines/qwen3_8b_kv/engine.py。

16 状态保存与恢复的完整过程

8B主机存储组件保存每层K/V，以及用于决定下一步输出的logits。权重继续由模型引擎持有。一次保存完成后，会话GPU状态可以释放；再次访问该会话时，恢复组件提供新的GPU对象，由消费者继续推进。

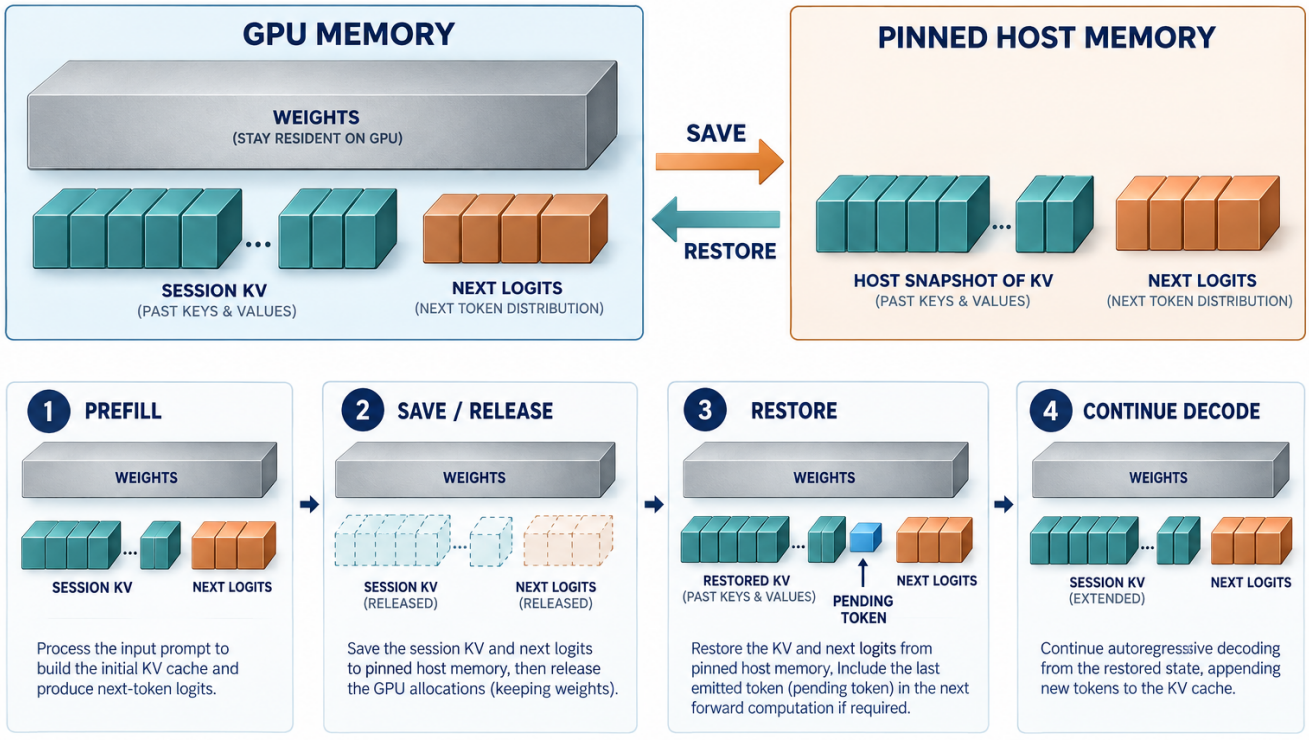


图3 GPU与锁页主机内存之间的会话状态流动。权重保持驻留；保存对象包括KV和下一步logits。会话续写时还要正确处理尚未进入KV的末token。

保存的提交点

主机快照需要在相关拷贝完成之后才对外提交。当前实现收集涉及的CUDA stream，在保存侧完成同步后更新已保存条目及容量统计。提交成功意味着调用者可以把它作为后续恢复的状态来源。

恢复的所有权

恢复操作为调用者创建独立结果，避免后续恢复或调用者修改覆盖已保存状态。对于同一CUDA stream，数据拷贝和后续计算按流顺序执行；跨stream使用时，需要由调用方建立相应依赖。消费者负责把这些对象接入正确的会话位置。

来源：engines/qwen3_8b_kv/kv_store.py；engines/qwen3_8b_sessions/consumer.py。

17 存储接口：明确值、身份与容量

主机KV组件提供简洁的值接口：保存、读取和删除。调用者不需要理解内部CPU张量怎样分配，但需要提供正确的模型身份，并遵守返回对象的使用顺序。接口越明确，构造器就越容易替换内部实现而保持行为一致。

接口	作用
KVStore(max_bytes)	创建有已提交载荷预算的存储对象
put(key, kv, logits, identity)	保存指定键的完整会话快照
get(key, identity, device)	验证身份并恢复到指定设备
delete(key)	删除条目并回收已提交载荷记账
used_bytes	查询当前已保存张量的载荷大小

模型身份属于状态语义

同一个键名不意味着两份缓存可以互换。权重、模型结构或精度变化后，旧KV通常不能直接交给新引擎继续使用。identity把这些条件与保存的状态联系起来，读取时核对身份，防止调用方把不匹配的快照接入推理。

容量与替换

保存新条目前，需要计算KV和logits的载荷。替换已有键时，容量判断应考虑旧值能够释放的部分；保存失败后，不能留下一个被计作成功的半快照。删除条目后，容量统计相应减少。调用者据此决定何时驱逐或关闭会话。

锁页主机内存

优化版本为GPU数据准备锁页CPU目标，通过拷贝接口保存张量。使用锁页内存，是为了改善主机与GPU之间的数据移动；同时仍然需要管理提交顺序、恢复副本和预算。当前max_bytes统计的是已提交张量载荷，进程实际内存还包括对象和临时分配，因此评测同时记录相应资源行为。

公开接口可以被单独复验，也可以作为联合消费者的存储后端。基础值接口保持稳定，场景专用的追加能力则作为可选接口扩展。

来源：engines/qwen3_8b_kv/kv_store.py；8B存储合同与host_roundtrip.py。

18 会话续写：最后一个token在哪里

生成最后一个输出token后，引擎通常不再执行下一次前向，因为当前请求已经结束。这样可以避免一次没有必要的计算，但也意味着最后这个token尚未进入KV。下一轮继续会话时，消费者必须先处理它。

设某会话已有一段输入，模型连续生成若干token。除最后一个输出外，前面的输出都已经作为后续步骤的输入进入KV。请求结束时，消费者保存“已经形成的KV”和“尚待消费的末token”。下一轮新增输入到来后，先把待消费token接到新增输入之前，再继续前向。

为什么这是消费者的职责

存储组件能够完整复制KV，但无法仅凭KV知道上层是否还有待消费的输出。如果消费者只保存张量、忽略执行位置，那么即使每个字节都正确恢复，继续生成仍可能与连续运行不同。完整会话语义需要由引擎状态与消费者状态共同表达。

会话时刻	GPU或主机中的状态	消费者需要记住的内容
首轮预填充后	输入对应KV、下一步logits	会话标识与已处理长度
生成过程中	已消费输出对应的扩展KV	已发出token数量
请求结束	末token之前的KV	pending_ids中的待消费token
会话驱逐	可恢复快照	待消费token仍属于该会话
下一轮开始	恢复或继续使用原KV	先消费pending，再接新增输入

公开SessionRuntime提供turn、close和stats接口。turn负责推进某个独立会话；close释放对应状态；stats返回预算和运行统计。通用LRU消费者与生成消费者在同一轨迹下运行，逐输出核对，使这种位置语义能够在不同实现之间保持一致。

这也是存储与推理联合设计的一个直接例子：可恢复对象不只有张量，还有“下一步应当做什么”的执行信息。

来源：engines/qwen3_8b_sessions/consumer.py；experiments/qwen3_8b/baseline_consumer.py。

19 联合优化：在驱逐时同步，按后缀追加

主机恢复组件解决了怎样保存与恢复，消费者进一步决定何时执行这些操作。如果每次会话turn结束都保存完整快照，即使会话随后仍驻留GPU，也会发生一次主机同步。会话不断增长时，已经保存过的前缀还可能被重复传输。

联合构造的第二轮围绕这两个问题调整了配合方式：把早期候选每轮结束后的主机同步推迟到实际驱逐时，同时保留已提交前缀，接受后续增长的后缀。通用LRU基准同样在驱逐时保存；两者进一步的区别，在于恢复后是否保留主机副本，以及下次同步是否需要重传完整状态。

驱逐时同步

GPU会话预算需要腾出空间时，消费者选择要驱逐的会话，将其当前状态提交给主机存储，再释放GPU引用。只要会话仍然驻留并被继续访问，就可以在GPU上推进，减少过早的备份。

保留前缀与追加后缀

模型继续生成时，旧KV前缀保持原值，新token增加后缀。当前冻结引擎以新张量追加状态，满足这种使用方式。生成存储据此维护版本化主机前缀，并在同步时只接收增长部分和最新的下一步logits。版本句柄帮助双方确认当前操作对应哪个前缀和位置。

配合点	基准LRU消费者	生成的联合做法
同步时机	GPU预算触发驱逐时保存	GPU预算触发驱逐时同步
主机副本	恢复后删除，后续重新保存	恢复后保留前缀供后续追加
增长状态	下次驱逐保存完整快照	复用主机前缀并追加后缀
接口选择	put/get/delete	支持可选增量能力，并保留基础接口

这些变化减少了组件统计中的主机载荷提交量，但最终是否加快会话，需要由完整轨迹判断。项目将存储和消费者交叉组合，观察单独替换与配套替换的不同结果，而不把字节减少直接等同于总时延收益。

来源：engines/qwen3_8b_sessions/kv_store.py与consumer.py；results/qwen3-8b-sessions-20261031.json。

20 实验设计：从可运行到可比较

实验围绕两个问题展开。第一，构造器能否生成符合接口与行为要求的运行时。第二，这个运行时在明确的模型、设备和负载下，表现如何。两类问题需要分别记录构造成本和执行结果。

0.5B实验以本项目初版运行时为起点，真实调用Agent生成批量解码候选，再以相同请求和检查方式测量。8B实验先冻结模型消费者，单独生成和优化主机存储；随后共同生成存储与会话消费者，并将组件交叉组合。

实验	固定条件	主要观察
0.5B构造与批量解码	单A40、float32、四请求、每请求16输出	构造成本、正确性、同条件吞吐
8B主机状态恢复	Qwen3-8B、BF16、冻结引擎、五档前缀长度	快照语义、恢复耗时、继续生成耗时
8B多会话组合	六会话24轮、每轮4输出、固定GPU与主机预算	完整输出、生命周期、四格总时延
发布验收	v0.2.0实际源码与固定0.5B权重	安装、服务、并发、文本与参考输出

预热、重复与原始记录

性能测量在预热后进行，并保留每次重复的原始值。0.5B使用A-B-B-A顺序比较初版与生成候选，减少单次运行顺序的影响。8B会话实验在每种组合计时前完整预热，并保留参考、预热、测量和驻留控制记录。

对完整请求负责

恢复微基准可以回答一个存储操作是否更快；“恢复后生成16 token”进一步回答对继续推理的影响；24轮会话轨迹则检验组件配合对整个场景的影响。三种时间口径分别保留，使结果能够支持对应层次的结论。

本作品的主要公开结果来自固定开发负载。结果表中明确列出比较对象、统计口径和硬件条件，便于使用者判断是否适合自己的请求，再用自己的轨迹继续验证。

来源：docs/EXPERIMENTS.md；results/campaign-20261030.json；8B公开结果JSON。

21 实验一：真实构造批量解码引擎

2026年10月30日，项目从已有0.5B运行时出发，真实调用Coding Agent修改engine.py与server.py。一次调用用时约77.7秒，产生支持活跃请求批量解码的候选，并通过固定验收。运行条件为Qwen2.5-0.5B-Instruct、单A40、float32，四个并发请求各生成16个token。

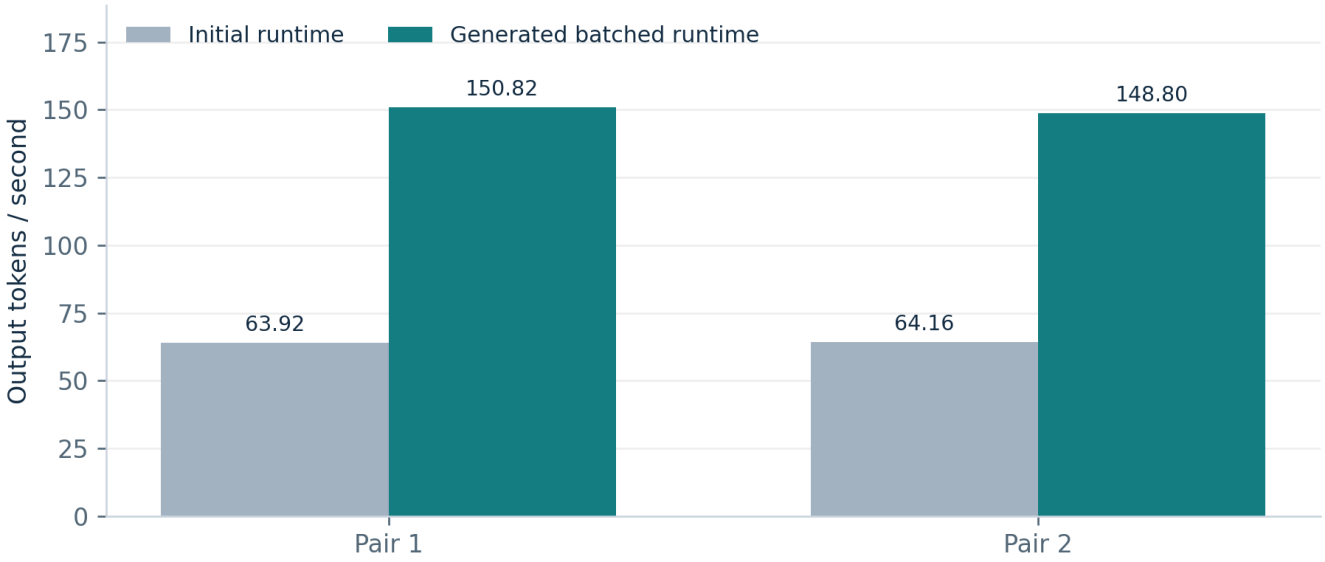


图4 同一开发负载下的初版运行时与生成候选。每次运行包含预热和三次测量，图中展示各次中位吞吐。

实现	第一次中位吞吐	第二次中位吞吐
本项目初版逐请求运行时	63.92 token/s	64.16 token/s
Agent生成的批量解码候选	150.82 token/s	148.80 token/s

按配对比值计算，几何平均约为2.34倍。代码变化与机制能够对应起来：服务把活跃请求组织成批，引擎提供相应的batch_step，同时保留每请求状态与文本输出。这个结果体现的是针对已知短请求并发场景的具体改进。

发布时又从公开源码启动服务，检查普通与流式输出、四请求并发、参数限制和完整词表参考比较。构造记录说明代码如何产生，发布复验则说明用户拿到的公开版本能够执行相应路径。

来源：results/campaign-20261030.json；results/release-v0.2.0-validation.json。

22 实验二：8B主机恢复路径优化

8B存储实验先固定模型引擎，把可编辑范围集中到主机KV组件。构造器先生成可运行存储，再根据实际恢复时间和检查结果继续修改。两轮生成调用分别约58.7秒和80.8秒，优化版本采用锁页CPU缓冲和相应拷贝路径。

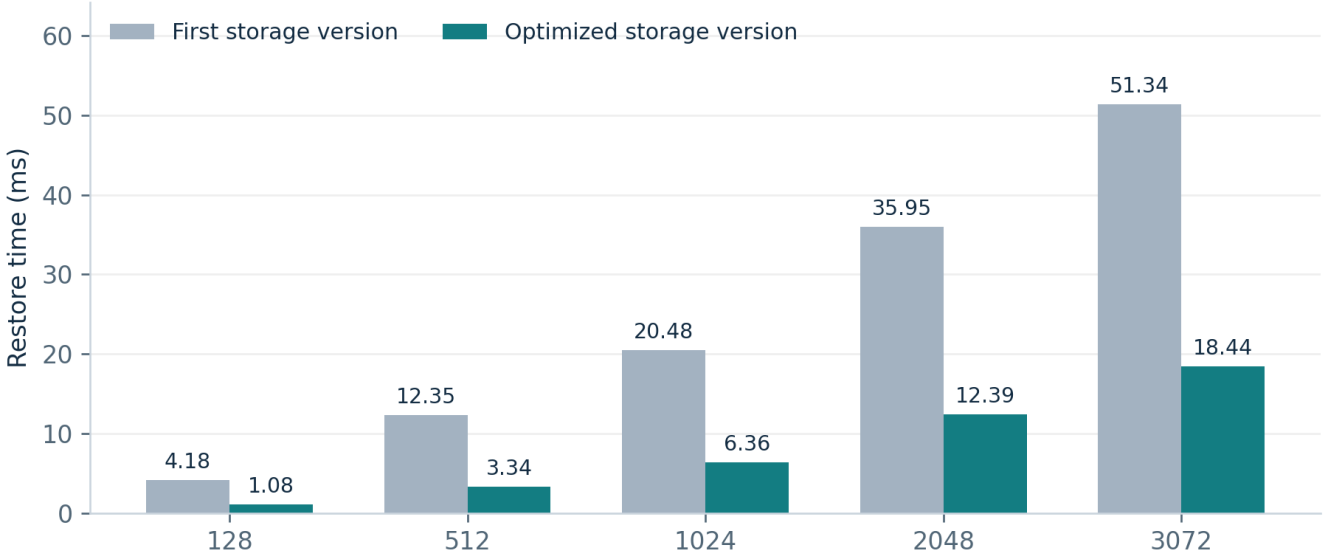


图5 不同前缀长度下的主机状态恢复时间。比较对象为本项目首版与反馈优化后的组件，使用同一冻结8B消费者。

输入token数	首版恢复	优化后恢复	首版/优化后
128	4.184 ms	1.082 ms	3.87×
512	12.346 ms	3.342 ms	3.69×
1,024	20.485 ms	6.359 ms	3.22×
2,048	35.947 ms	12.393 ms	2.90×
3,072	51.342 ms	18.442 ms	2.78×

每个版本都经过九组状态检查，覆盖不同长度、预填充和中途解码保存，以及batch1与batch2。恢复后继续生成，并核对完整logits与token。这使恢复时间的变化能够对应到保持原有状态语义的实际实现。

前缀越长，需要恢复的KV载荷越大，两个版本的恢复时间都随之增加。优化后的绝对时间和比值分别展示了改善幅度；后续还需要将这种变化放回完整续算流程，观察它在总体时间中占多大比例。

来源：results/qwen3-8b-host-state-20261031.json；experiments/qwen3_8b/provenance.json。

23 从恢复操作到恢复后继续推理

存储操作变快之后，用户实际等待的时间会减少多少？项目在恢复微基准之外，还测量了“恢复状态后再生成16个token”的完整过程。两组结果共同说明存储优化在推理流程中的位置。

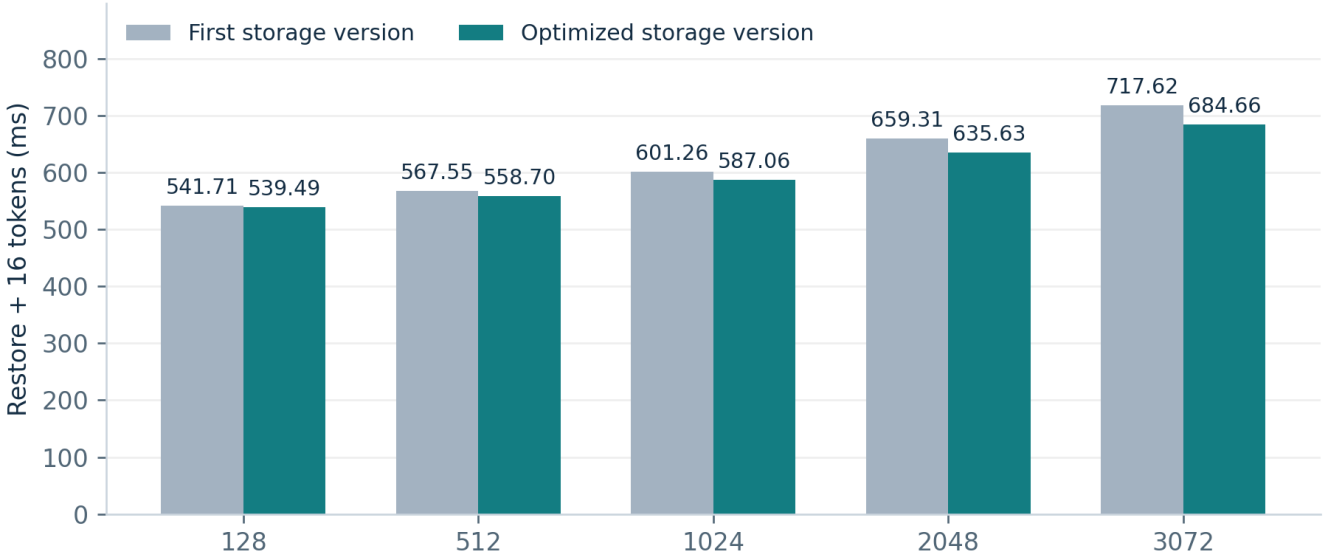


图6 恢复并继续生成16个token的总时间。与前页恢复微基准采用相同五档输入长度，单位为毫秒。

输入token数	首版总时间	优化版总时间	时间降幅
128	541.712 ms	539.495 ms	0.409%
512	567.545 ms	558.702 ms	1.558%
1,024	601.259 ms	587.064 ms	2.361%
2,048	659.311 ms	635.628 ms	3.592%
3,072	717.617 ms	684.659 ms	4.593%

恢复加速2.78—3.87倍后，完整续算时间下降约0.4%—4.6%。原因可以从时间量级理解：小前缀恢复只需几毫秒，而后续16步解码仍需数百毫秒；前缀增加后，恢复占比提高，总时间改善也随之扩大。

这一结果指导了下一步设计：继续关注单次拷贝的同时，需要让消费者减少重复同步、保留可复用前缀，并检查计算路径本身。优化目标由“把一个存取操作做快”进一步落到“把整次会话推进做得更合适”。

来源：results/qwen3-8b-host-state-20261031.json。

24 实验三：存储与消费者的四格组合

联合实验使用六个合成会话，共24轮，每轮输出4个token。初始输入长度依次为512、1024、2048、3072、2048和1024。完成轮次时，会话GPU状态预算为768MiB，主机已提交载荷预算为4GiB。每种组合先完整预热，再进行三次测量。

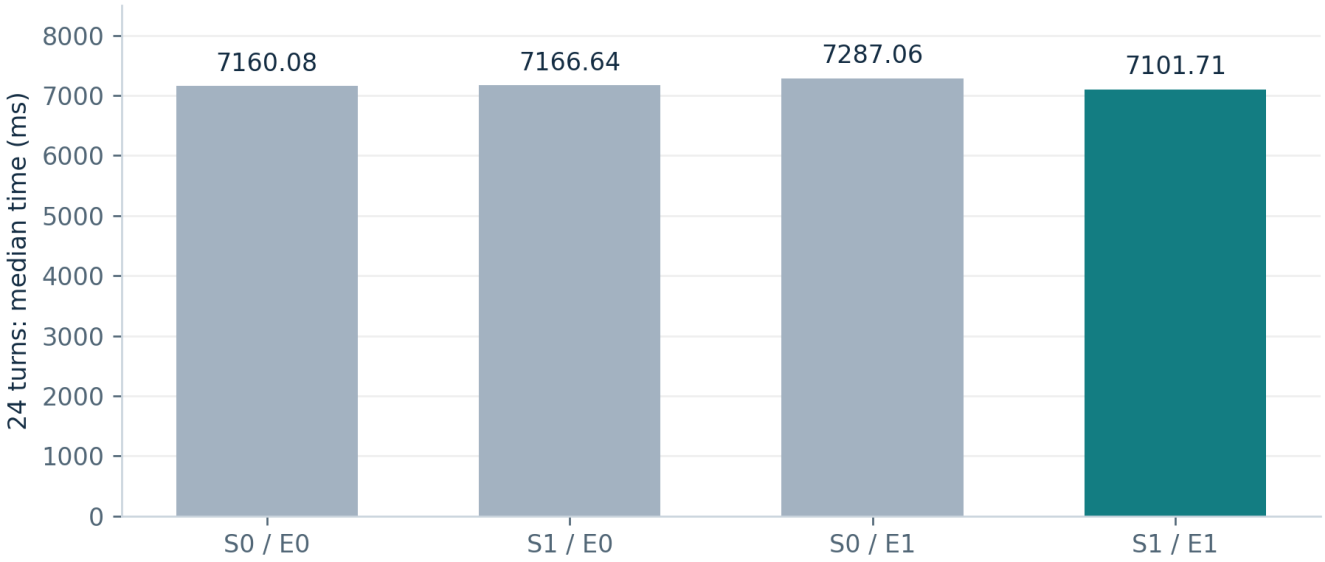


图7 24轮会话的耗时中位。S0为基准存储，E0为通用LRU消费者；S1与E1为共同生成的存储和消费者。时间包含逐输出观测。

组合	24轮耗时中位
S0 / E0：基准组合	7,160.08 ms
S1 / E0：只换存储	7,166.64 ms
S0 / E1：只换消费者	7,287.06 ms
S1 / E1：配套替换	7,101.71 ms

配套组合相对基准中位下降0.815%。单独替换存储或消费者没有得到相同表现，说明接口能力与调用时机需要共同观察。这一差值较小，本轮将它作为后续优化线索。

公开记录包含1次参考、4次预热、12次计时和1次驻留控制，共18条完整轨迹、432轮请求、1728个输出位置。所有输出按同一冻结引擎的无换出参考核对，会话关闭后检查主机和GPU状态回收。程序按张量大小累计的逻辑写回载荷由约4.00GB降到1.49GB，提供了减少重复同步的直接解释。

来源：results/qwen3-8b-sessions-20261031.json；experiments/qwen3_8b/session_matrix.py。

25 工程质量：让状态和失败可检查

生成式开发仍然需要明确的软件质量要求。对推理和存储代码，最重要的问题常常发生在成功路径之外：恢复对象是否共享了错误的存储，容量不足时旧值是否保留，会话关闭后是否还有隐藏GPU引用，以及请求被中断时状态是否及时结束。

项目把这些要求写入接口检查和实际执行流程。主机状态测试在预填充和解码中途保存快照，恢复后继续运行；独立负例检查存储实现是否偷偷保留了GPU副本；会话评测在每轮和所有会话关闭时记录资源。输出比较与生命周期检查一起决定候选能否进入后续测量。

检查对象	具体观察	工程价值
保存完整性	每层K/V和下一步logits	防止只保存了部分状态
恢复副本	修改恢复对象后重新读取	检查主机快照与调用者对象隔离
设备引用	释放会话后检查GPU张量	识别隐藏的显存保留
身份和版本	模型身份、前缀和句柄代次	防止旧状态被错误复用
会话结束	close后主机payload与GPU状态	验证资源生命周期
服务输入	长度、类型、重复请求标识	让调用失败有明确返回
文本输出	中文与emoji边界、SSE顺序	保证客户端接收完整结果

小例子优先于长时间盲测

当请求交错出现错误时，先缩小能够保留原失败的请求集合，比反复运行大负载更有价值。项目的请求缩小工具保留失败签名，在删除外部请求后重新检查问题是否仍然存在。缩小后的失败样例更适合反馈给构造器，也便于后续回归。

质量记录与代码版本一同保存。新增优化首先回答对应检查是否通过，再解释性能变化；已有正确实现也可以持续接受新的边界用例，逐步扩大可使用范围。

来源：forge/replay.py；8B tests目录；host_roundtrip.py；session_matrix.py。

26 代码组织与版本交付

公开仓库按构造工具、运行产物、任务与实验组织。使用者可以从README和快速运行指南进入服务，也可以直接打开某个8B组件，查看它的接口、合同和对应结果。目录结构让代码与解释尽量保持相邻。

目录或文件	内容
forge/	构造控制器、种子引擎、服务、评测与失败缩小
examples/generated/batched/	实际生成的0.5B批量解码引擎与服务
engines/qwen3_8b_kv/	冻结8B引擎及主机状态存储
engines/qwen3_8b_sessions/	生成的配套存储与会话消费者
experiments/qwen3_8b/	独立构造、接口检查、恢复及会话矩阵
tasks/ 与 targets/	任务设置、模型和硬件目标
results/	可公开回读的实验与发布验证结果
docs/	架构、运行、演示和提交材料

从源码身份到发布版本

评测记录中的源码哈希用于绑定具体实现，Git提交和发布tag用于交付版本。当前运行代码以v0.2.0为公开基线，包含可直接启动的0.5B服务和独立8B实验入口。模型权重按固定revision下载，不随代码仓库重复分发。

发布时实际做了什么

v0.2.0在独立目录安装并构建，25项CPU检查通过。随后在真实A40上运行服务，检查普通HTTP、中文SSE、四请求并发和输入限制；对生成引擎进行12次完整词表参考比较。生成代码的两个文件哈希与公开结果记录绑定，便于使用者确认自己运行的版本。

主分支通过GitHub CI，展示页由GitHub Pages发布。源码、PDF与视频分别提供稳定入口，使用说明进一步给出模型准备、启动和检查命令。交付的重点是让其他人能够沿着同一条路径把项目运行起来。

来源：公开仓库v0.2.0；.github/workflows/check.yml；发布验证记录。

27 快速运行：从安装到第一条流式输出

已有引擎可以直接运行。准备Linux、NVIDIA GPU和Python环境后，从公开版本克隆代码，安装推理依赖。下面使用仓库的0.5B服务作为最短体验路径。

```
git clone --branch v0.2.0 \
  https://github.com/yanchaomei/inference-forge.git
cd inference-forge
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e '.[inference,test]'
```

从Qwen官方仓库准备Qwen2.5-0.5B-Instruct，固定revision为7ae557604adf67be50417f59c2c2f167def9a775。完整下载命令在docs/QUICKSTART.md中给出。选择空闲GPU，将模型路径传给服务：

```
export CUDA_VISIBLE_DEVICES=0
export FORGE_MODEL_PATH=/path/to/Qwen2.5-0.5B-Instruct
python -m examples.generated.batched.server \
  --model "$FORGE_MODEL_PATH" --dtype float32 \
  --port 8000 --event-log runs/demo-events.jsonl
```

看到ready后，在另一终端检查健康状态并发送请求：

```
curl --fail http://127.0.0.1:8000/health
curl --fail -N http://127.0.0.1:8000/v1/completions \
  -H 'Content-Type: application/json' \
  -d '{"prompt":"The capital of France is",
       "max_tokens":16,"stream":true}'
```

流式请求逐步返回token事件，最终以[DONE]结束。普通请求把stream设为false或省略，即可获得完整text、token_ids与usage。事件日志保留请求接收、调度和完成信息，适合演示后查看执行过程。

远程GPU可用SSH端口转发连接本地客户端。0.5B服务的总上下文上限为512 token，输出上限为64 token；运行8B实验时使用下一页的独立入口。

来源：docs/QUICKSTART.md；examples/generated/batched/server.py。

28 8B复现：存储检查与会话组合

8B实验使用独立依赖环境。公开requirements固定了已验证的软件组合，模型准备工具下载指定revision并核对官方文件身份。使用者先确认模型资产，再运行组件检查和完整会话实验。

```
python3 -m venv .venv-8b
.venv-8b/bin/python -m pip install \
  -r experiments/qwen3_8b/requirements.txt
export FORGE_MODEL_PATH=/path/to/Qwen3-8B-b968826
export FORGE_GPU_UUID=GPU-your-id
export FORGE_PYTHON="$PWD/.venv-8b/bin/python"
```

模型采用Qwen3-8B的b968826d9c46dd6066d109eabc6255188de91218版本。准备工具下载模型并校验官方文件身份：

```
"$FORGE_PYTHON" scripts/prepare_model.py \
  --target targets/qwen3-8b-a40/target.json \
  --metadata targets/qwen3-8b-a40/model-files.json \
  --destination "$FORGE_MODEL_PATH"
```

单独检查主机保存与恢复：

```
PYTHONPATH=. "$FORGE_PYTHON" \
  -m experiments.qwen3_8b.host_roundtrip \
  --model-path "$FORGE_MODEL_PATH" \
  --gpu-uuid "$FORGE_GPU_UUID" \
  --output runs/host-state-check --correctness-only
```

比较四种存储与消费者组合：

```
PYTHONPATH=. "$FORGE_PYTHON" \
  -m experiments.qwen3_8b.session_matrix \
  --model-path "$FORGE_MODEL_PATH" \
  --gpu-uuid "$FORGE_GPU_UUID" \
  --candidate engines/qwen3_8b_sessions \
  --output runs/session-evaluation --repeats 3
```

生成新候选时，在本地已登录Codex CLI的环境使用construct入口，指定账户可用的生成模型和新的输出目录。生成结束后，把候选同步到GPU环境，用相同评测命令替换candidate路径。检查结果、代码身份和原始输出保存在对应运行目录中。

来源：experiments/qwen3_8b/README.md、construct.py、requirements.txt。

29 开源协作：让任务与经验能够复用

项目采用MIT许可证发布代码。用户可以阅读、运行和修改已有实现，也可以提交新的任务、设备适配、存储组件和可复现问题。模型权重来自模型发布者，按其许可证单独下载。

从一个小而完整的贡献开始

新的模型或硬件适配，首先应提供一个可以完整执行的任务：固定模型身份，给出代表性输入，说明正确输出和资源要求，并附运行命令。性能贡献则同时提供原始测量、预热与重复方式，使其他人可以沿相同步骤比较。

对于生成代码，构造提示、起始源码和调用预算同样重要。它们解释了代码如何产生，也让后来者能够复用有效反馈、继续修复失败。只提交最终文件，容易丢失这些有价值的研发信息。

贡献方向	推荐提交内容
模型适配	配置、权重身份、前向实现与参考检查
硬件适配	执行环境、设备能力、对应任务与原始结果
存储组件	基础值接口、生命周期说明与恢复检查
调度或消费者	状态推进规则、组合测试和请求轨迹
问题报告	版本、复现命令、尽量缩小的失败输入与实际输出
使用文档	可执行步骤、预期观察和必要依赖

与已有开源工作的关系

项目吸收VibeServe、MetaInfer等工作的生成与执行反馈思路，也借鉴BespokeOLAP按场景共同组织数据表示和执行逻辑的方式。InferenceForge将问题集中到推理状态与执行代码，持续通过实际构造和公开结果检验实现。

开源协作的目标，是逐步积累能运行的任务、能检查的接口和能解释的结果。每增加一个高质量任务，就为下一次模型或设备适配提供一份可继续使用的基础。

来源：CONTRIBUTING.md；NOTICE.md；docs/SPECIALIZATION.md。

30 核心特点与后续技术路线

InferenceForge的特点可以归纳到三个具体设计选择。它们分别作用于代码怎样产生、状态怎样组织，以及成果怎样交付。

按场景构造实际代码

输入模型、硬件与负载后，构造器可以修改引擎和服务实现。0.5B实例中的批量解码就是代码级变化：服务改变了活跃请求的组织，引擎增加了匹配操作，固定评测验证其数值和请求行为。专用化因此能够进入执行路径，而不只停留在外部配置。

把数据存取与消费方式一起设计

8B联合实现把主机副本保留多久、何时同步后缀、恢复后由谁继续消费，作为同一个问题处理。驱逐时同步、版本化前缀和pending token语义都来自真实会话过程。这个方向使存储优化能够与计算路径和请求生命周期相互解释。

将执行反馈保留为工程资产

任务合同、固定检查器、源码身份和原始运行结果，构成了可继续复用的研发环境。使用者可以复验已有代码，也可以替换生成器或提供人工实现，用同样的条件检查。工具链与运行产物分别交付，便于持续迭代和独立使用。

后续工作	需要回答的具体问题
服务性能	去除完整观测开销后，真实HTTP负载的吞吐与时延怎样变化
自然会话轨迹	重访间隔和上下文增长改变后，何时保存与恢复更合适
同预算构造比较	相同起点、生成模型与预算下，反馈组织怎样影响构造效率
更丰富的执行实现	哪些实际计算或数据路径值得进一步融合和特化
模型与硬件迁移	已有任务、接口和反馈能复用多少，哪些条件需要重建

阶段性结果已经提供了可运行起点：0.5B构造得到明确的批处理改善，8B恢复组件完成实际优化，多会话实验把组件收益放回完整流程。后续研发沿这些可执行对象推进，用新的负载和同条件对照扩大项目价值。

来源：公开源码与实验记录；docs/SPECIALIZATION.md中的后续安排。

31 版本、资料与提交入口

作品名称：InferenceForge，面向模型、硬件与场景的专用推理引擎构造工具链。参赛方向：开源基础软件与解决方案。单位：中国科学技术大学。联系人：闫超美，yanchaomei@mail.ustc.edu.cn。

公开使用入口

项目	地址
代码仓库	https://github.com/yanchaomei/inference-forge
运行代码基线	v0.2.0，提交03b8a0a
在线展示与视频	https://yanchaomei.github.io/inference-forge/
视频文件	https://yanchaomei.github.io/inference-forge/demo.mp4
作品介绍PDF	https://yanchaomei.github.io/inference-forge/project-introduction.pdf

主要证据与阅读资料

资料	用途
README.md、docs/QUICKSTART.md	项目入口与0.5B实际运行步骤
docs/ARCHITECTURE.md、docs/SPECIALIZATION.md	组件职责与特化路线
results/campaign-20261030.json	0.5B构造与A-B-B-A数据
results/qwen3-8b-host-state-20261031.json	五档恢复和续算时间
results/qwen3-8b-sessions-20261031.json	四格组合与432轮逐输出记录
results/release-v0.2.0-validation.json	安装、服务和参考输出复验
experiments/qwen3_8b/README.md	8B构造、接口检查与复现步骤

本说明的模型结构、接口行为和数值来自对应源码与公开记录。0.5B实验使用float32；8B状态实验使用Qwen3-8B、BF16及单A40。不同实验的时间口径在对应图表中说明。

演示视频为2026年10月30日实际0.5B构造与运行记录的90秒字幕回放，等待已压缩。本文同时展开8B存储和消费者的代码及实验，读者可按独立入口继续复现。

封面与三张机制图使用图像生成工具制作，并按源码关系校对；四张统计图直接读取公开JSON绘制。文稿、图像提示、原始图、排版程序和最终文件校验记录一并保存，支持后续继续编辑与版本更新。